Wichtige Funktionen zur Auswertung von Versuchen.

Diese Vorlage kann kopiert werden und dann für den jeweiligen Versuch genutzt werden.

Zunächst soll die Versuchsummer des jeweiligen Versuches angegeben werden, damit das automatische Speichern der Grafiken in das richtige Verzeichnis funktioniert.

Dieses Dokument ist an vielen Stellen (stark) an *Luca Hafner* orientiert.

Versuchsnummer:

In [1]:
```
versuchsnummer = "221"
```

Aufgabe dinamisch ändern, damit das automatische Speichern besser funktioniert.

In [2]:
```
aufgabe = None
```

Import aller wichtigen Libaries

In [3]:
```
import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms
%matplotlib inline

import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial

import os
import os.path

import pandas as pd
import csv
import re
```

```python
from typing import List

import math
from decimal import Decimal, ROUND_HALF_UP, getcontext

import pylab as py
from IPython.display import display, Math, Latex, HTML

import sympy as sp
from sympy import separatevars
```

Erstellen der dedizierten Dartei zum Speichern der Werte für **LATEXT**

Folgender Code sollte einmal ausgeführt werden, damit direkt die Datei zum Speichern erstellt werden kann. Unbedingt daran denken, die Versuchsnummer anzupassen, die Werte anderer Versuche könnten ansonsten verloren gehen. Es gibt jedoch eine sicherheits Kopie.

Diese beinhaltet:

- Formel, sowie Formel nach gff
- Berechnete Werte und deren Fehler
  - Wert + Fehler
  - Tabellen Export

In [4]:
```python
def create_tex_result_values(fileName = "python-results.sty"):
    """
    Diese Method erstellt automatisch eine tex-Datei, in dem Messwerte bzw. deren E
    Die File wird unter *Versuche/${versuchsnummer}$/Auswertung/python-results.tex*
    angepasst werden muss.

    Die Python-File muss im Python-Ordner des jeweiligen Versuches liegen!

    Parameter
    ----------
    **fileName**:
        neuer Name, falls die Datei besonders heißen soll.
    """
    # Erstellung der file

    path = "../Auswertung/" + fileName

    if os.path.isfile(path):
        pass
    else:
        with open(path, 'w') as file:
            file.write("% Dies ist eine automatisch generierte Datei. Hier werden a

        with open(path + "-BackUp.sty", 'w') as bFile:
            bFile.write("% Dies ist eine automatisch generierte Datei. Hier wird de
```

```
    # print(filePath + "/" + fileName)
    # print(versuchsnummer)

    return path

create_tex_result_values()
pyPath = create_tex_result_values()
```

Definition der Funktionen

Table of Contents:

- Fehlerrechnung einer gegebenen Formel
  - Gaus Fehlerhfortpflanzung (gff))
  - Runden auf signifikante Stellen
- Automatisierter Latex-Export
  - Export der Latex Formel
  - Ergebnisse plus Fehler
  - Export als Tabelle

Fehlerrechnung einer gegebenen Formel

Gaus Fehlerhfortpflanzung (gff):

Die *Methode gff* nimmt zwei Argumente, zum einen eine *sympy (sp)* Funktion, zum anderen einen Array aller fehlerbehafteter Größen.

In [5]:
```
def gff(func, errPronePar, params_without_error=None):
    """
    Führt standardgemaess die Gaussian Error Propergation aus.
    """
    # Variablen definieren
    error = 0
    errProneParamters = []
    if params_without_error is None: params_without_error = []
    for errPar in errPronePar:
        if errPar in params_without_error:
            continue
        d = sp.symbols('thisWillTurnDelta' + errPar.name)
        partial = sp.diff(func, errPar) * d  # Die Funktion wird nach der fehlerbeh
        error = error + partial**2 # Fehler werden quadratisch aufsummiert
        errProneParamters.append((errPar,d))
    absolut_err=sp.simplify(sp.sqrt(error),rational = True)
    relativ_err=sp.simplify(sp.sqrt(error/func**2),rational = True)
    return absolut_err, relativ_err, errProneParamters
```

Runden auf signifikante Stellen

In [6]:
```
def round_with_carry(number_str):
    number_list = list(number_str)

    i = len(number_list) - 1

    # Solange 9 auf 0 setzen und nach links gehen
    while i >= 0 and int(number_list[i]) > 4:
        number_list[i] = '0'
        i -= 1

    # Wenn wir noch links eine Stelle haben erhöhen
    if i >= 0:
        number_list[i] = str(int(number_list[i]) + 1)
    else:
        # Alle Stellen waren 9 neue 1 vorne
        number_list.insert(0, '1')

    return ''.join(number_list)

def round_sig_digs(errVal: float, val: float=None):
    """
    Function that rounds error value accoring the the standardised format:
    the function will search for the first digit that is greater than 2. If it foun
    It will than present the value according to the errVale, thus they have the sam

    Parameter
    ---------
    **errValue** : float to be rounded to significant digit.

    **value** : float that will be rounded accoring to the errValue.

    Returnes
    --------
    rounded : float
    """

    # print(f"Messwert: {val}, mit Ungenauigkeit: {errVal}")

    errVal = Decimal(str(errVal))
    val = Decimal(str(val))
    err_str = f"{errVal.normalize():f}"
    if '.' in err_str:
        err_int, err_frac = err_str.split('.')
    else:
        err_int = err_str
        err_frac = ''
    dez_pos = len(err_int) # Position des Kommas
    err_comb = err_int + err_frac + '0' # Alle Ziffern des Fehlers ohne Dezimal-tre
    err_diffrent_dig = len(set(list(err_comb))) # Gibt die Anzahl der verschiedenn

    leadZeros = False
    sig_dig_bool = False
    numZeros = 0
```

```
non_sig_dig = ""

rounded_err_dig = ""
sig_dig_nine = False
# ---------------------------------------------------------------------------
#   Untesucht ob und wo die Signifikante ist.
# ---------------------------------------------------------------------------

for numPos, dig in enumerate(err_comb):
    if int(dig) > 2: # Schaut ob dig sig ist.
        pos_sig_dig = numPos
        sig_dig_bool = True
        if dig == '9':
            sig_dig_nine = True
            # print(f"Anzahl aufeinanderfolgender 9en: {nine_count}")
        break

    elif int(dig) == 0:
        leadZeros = True
        numZeros = numZeros + 1
        non_sig_dig = non_sig_dig + dig
    else:
        non_sig_dig = non_sig_dig + dig
        pass

if leadZeros == True and sig_dig_bool == False:
    if (('1' in err_comb or '2' in err_comb)) and (err_diffrent_dig== 1) or (('
        # print("Alle Ziffern sind nicht signifikant")
        pass

    elif (('0') in list(err_comb)) and (len(set(list(err_comb)))== 1):
        # print("Die Ungenauigkeit ist null")
        pass

    else:
        # print("Alle stellen sind 0, 1 oder 2 ")
        pass

# ---------------------------------------------------------------------------
#   Untersucht die Folgestelle der Signifikanten Stell: Auf- oder Abrunden?
# ---------------------------------------------------------------------------

# Wenn signifikante Stelle, dann entsprechend Runden:
if (sig_dig_bool == True):
    if (int(err_comb[pos_sig_dig + 1]) < 5):
        rounded_err_dig = int(dig)
        rounded_up = False
    else:
        rounded_err_dig = int(dig)  + 1
        rounded_up = True

if sig_dig_bool == False:
    # Wissenschaftliche Schreibweise mit e fixen
    if 'e' in str(errVal):
        res = Decimal(str(errVal))
    else:
```

```
            rounded_err = non_sig_dig
            a = list(rounded_err)
            a.insert(dez_pos, '.')
            res = Decimal(''.join(a))

    # Neunen richtig runden
    elif sig_dig_nine and rounded_up:
        rounded_err = round_with_carry(err_int)
        res = rounded_err

    # Nullen ordentlich betracten
    elif leadZeros == True and err_int == "0" and sig_dig_bool == True:
        rounded_err = non_sig_dig + str(rounded_err_dig)
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = ''.join(a)

    elif dig in err_int:
        rounded_err = non_sig_dig + str(rounded_err_dig) + (len(err_int) - pos_sig_
        res = rounded_err
    else:
        rounded_err = non_sig_dig + str(rounded_err_dig)
        a = list(rounded_err)
        a.insert(dez_pos, '.')
        res = ''.join(a)

    # print(f"gerundeter Fehler: {res}")

    # ---------------------------------------------------------------------------
    #   Anpassen des Messwertes an den gerundeten Fehler (Gleich gerundet)
    # ---------------------------------------------------------------------------

    val = Decimal(str(val))
    res = Decimal(str(res))

    # Exponent (z.B. -2 bei 0.12, +2 bei 200)
    num_decimals = -res.normalize().as_tuple().exponent

    # Rundungsstelle bestimmen
    if sig_dig_bool:
        rounded_val = val.quantize(Decimal('1.' + '0' * (num_decimals)), rounding=R
    else:
        rounded_val = Decimal(str(val.quantize(Decimal('1.' + '0' * (num_decimals))


    # print(f"Gerundeter Wert: {format(rounded_val)}")
    # print(f"Ergebnis: {format(rounded_val, 'f')} \\pm {format(res)}")
    # print("---------------- \n")

    return rounded_val, res
```

Export einer Funktion in Latex

Die folgende Funktion transformiert eine Formel in Latex. Diese Formel kann entweder hier im Notebook einfach kopiert werden oder als Variable in der *pyton-results.tex*.

In [7]:
```
def function_to_latex(f, texVarName, texCom):
    """
    Zeigt die Formel als gerenderte Math-Darstellung und darunter
    den Latex-Quelltext, der per Button kopiert werden kann. (Für leichtere Benutzu

    Zudem wird die Latexformel als Variable in Latex gespeichert.

    Parameters
    ----------
    **f** : sympy function

    **texVarName** : Einzigartiger Name der Variablen. Dieser wird zum überschreibe

    **texCom** : setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash!
    """
    # print("Die gegebene Funktion lautet: \n")
    display(Math(sp.latex(f, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Del

    # print("Hier ist der dazugehörige Latex code: \n")
    latex_str = sp.latex(f, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delt
    html = f"""
    <div style="margin-top:0.5em;">
        <code id="latex-code-{id(f)}">
            {latex_str}
        </code>
        <button onclick="
            const tex_as_txt = document.getElementById('latex-code-{id(f)}').innerT
            navigator.clipboard.writeText(tex_as_txt)
        " style="
            margin-left:8px;
            padding:2px 6px;
            cursor:pointer;
        ">
            Kopieren
        </button>
    </div>
    """
    display(HTML(html))


    # Hinzufuegen bzw. Ueberschreieben der Formel in die Sammlung
    with open(pyPath, 'r') as file:
        lines = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines, 1):
            if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben
                # print(f'{texVarName} is at line {lineNum}')
```

```
                lines[lineNum] = "\\newcommand{\\" + texCom + "}{" + latex_str + "}
                found = True

    if not found:
        # print("Die neue Funktion wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n\n% " + texVarName + "\n") # Fügt die Variable als Kommen
            file.write("\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n")
    else:
        # print("Die alte Funktion wurde erfolgreich überschrieben.")
        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

Ergebnisse plus Fehler

In [8]:
```
def calc_with_err(func, errFunc, values):
    """
    Methode zum berechnen von Werten und deren Fehler.

    Parameter
    ----------
    **func** : sympy Funktion mit Parametern.

    **errFunc** : die zu func gehöhrende sympy Fehler-Funktion nach gff

    **values** :
        Werte, die in die Funktionen eingesetzt werden.
        Als array von Tupeln der Form [(a,da),(b,db),...] oder als array/liste [a,d
        (Reihenfolge muss die sein, in der die Argumente in der Funktion genommen w
    """

    #Falls der Input in mehrere Tupel aufgeteilt ist, werden diese zu einem Array z
    if (np.ndim(values) != 1):
        values = np.concatenate(values)
    result = func(*values[::2])
    uncertainty = errFunc(*values)
    return result, uncertainty
```

In [9]:
```
def pap(function, params, data, params_without_error=[]):
    # ,  texVarName = None, texCom = None
    """
    Nimmt als imput eine Funktion und die Information welche Parameter fehlerbehaft

    Parameter
    ----------
    **function** :
        sympy Funktion mit Parametern. In diese werden die Messwerte eingesetzt.

    **texVarName** :
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Forme

    **texCom** :
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch tex
```

```
    **params** :
        Parameter der Funktion. Diese werden als Array von Sympy-Symbolen gebraucht

    **data** :
        2D-Array mit den Messdaten, sodass die Zeilen die Form haben: [Parameter 1,
        Die Funktion wird zeilenweise angewandt. Wird kein Fehler für einen Paramet
        an die Funktion gegeben werden oder ganz weggelassen werden. Dann muss alle

    **params_without_error** :
        Alle Parameter zu denen kein Fehler explizit in den Daten angegeben ist. Di
        auch nicht in der Latex Form der Fehlerformel vor

    """
    # Expand data to include the uncertainty 0 for values with no assigned uncertai
    # Hat zubeginn die Form: [[0. 0. 0. 0. 0. 0. ...]].
    # Bei jeder Iteration werden dann die Parameter und deren Fehler hinzugefügt: i
    # exp_data = np.zeros((data.shape[0],data.shape[1]+len(params_without_error)))
    exp_data = np.zeros((data.shape[0], 2 * len(params)))
    i = 0      # läuft durch die Parameter
    j = 0      # läuft durch die expanded data
    z = 0      # läuft durch die eingegebene data, also die Messwerte und deren Feh
    # Läuft durch jeden Parameter und seinen Fehler
    while (i < len(params)):
        # Checkt, ob der Parameter Fehlerbehaftet ist. Wenn, dann wird an j-ter Ste
        if (params[i] in params_without_error):
            exp_data[:,j] = data[:,z]
            i = i + 1
            j = j + 2
            z = z + 1
        else:
            exp_data[:,j] = data[:,z]
            exp_data[:,j+1] = data[:,z+1]
            i = i + 1
            j = j + 2
            z = z + 2

    # z = 0  # index in original data

    # for i, param in enumerate(params):
    #     if param in params_without_error:
    #         exp_data[:, 2*i] = data[:, z]
    #         exp_data[:, 2*i+1] = 0
    #         z += 1
    #     else:
    #         exp_data[:, 2*i] = data[:, z]
    #         exp_data[:, 2*i+1] = data[:, z+1]
    #         z += 2


    # Create variable that stores parameters that have no assigned uncertainty
    params_with_error = []
    j = 0
    for n in np.arange(0,len(params)):
        if not (params[n] in params_without_error):
            params_with_error.append(params[n])
            j = j + 1
```

```
# Get the given function and error function as numpy functions
f = sp.lambdify(params,function)
absolut_err, relativ_err, parameters = gff(function,params) # Gauss Fehlerfortp
err_abs = sp.lambdify(np.concatenate(parameters),absolut_err)

args = []

# zuerst normale Parameter
for i, p in enumerate(params):
    args.append(exp_data[n, 2*i])

# dann Fehlerparameter
for i, p in enumerate(params):
    if p in params_without_error:
        args.append(0)
    else:
        args.append(exp_data[n, 2*i+1])

args = np.array(args)

results[n,:] = calc_with_err(f, err_abs, args)

# Calculate the results for each row of data
results = np.zeros((data.shape[0],2))
for n in np.arange(0,data.shape[0]):
    results[n,:] = calc_with_err(f, err_abs, exp_data[n,:])
    # print(calc_with_err(f, err_abs, exp_data[n]))

if (len(results) < 10):
    print("Aus den entstandenen Werten wurden folgende Ergebnisse Bestimmt:")
    print(results)

# Substitutes 0 for the uncertainty of the parameters without error, so it does
for p in params_without_error:
    absolut_err = absolut_err.subs('thisWillTurnDelta'+p.name,0)
for p in params_without_error:
    relativ_err = relativ_err.subs('thisWillTurnDelta'+p.name,0)

# Wiedergabe der Formeln in Latex
function = sp.simplify(function,symbols = params, rational= True)
function = sp.separatevars(function)


texVarName = "Place Holder Name"
texCom = "Place Holder Com"

print("Given values:")
print(exp_data[n,:])

print("gegebene Funktion:")
function_to_latex(function, texVarName, texCom)

print("-----------------------------------------------------------------\n")

print("Formel des absoluten Fehlers der gegebenen Funktion:")
```

```
    function_to_latex(absolut_err, "errAbs-" +  texVarName, "errAbs" +  texCom)

    print("-------------------------------------------------------------\n")

    print("Formel des relativen Fehlers der gegebenen Funktion:")
    function_to_latex(relativ_err, "errRel-" +  texVarName, "errRel" +  texCom)

    return(results)

def pap_now(function, params, data, params_without_error = []):
    length = 1
    for e in data:
        if(np.shape(np.array(e)) != ()):
            length = np.shape(np.array(e))[0]
            break
    data_2d = np.zeros((length,len(data)))
    for i in range (0,len(data)):
        data_2d[:,i] = data[i]
        return(pap(function,params, data_2d, params_without_error = params_without_
```

In [10]:

```
def pap_new(function, params, data, params_without_error=None):
    """
    Berechnet Funktionswerte mit Gaußscher Fehlerfortpflanzung.

    Parameters
    ----------
    function : sympy expression
        Funktion mit symbolischen Parametern.

    params : list of sympy symbols
        Parameter der Funktion in definierter Reihenfolge.

    data : array_like
        Messwerte in der Form:
        [p1, err_p1, p2, err_p2, ..., pn]
        oder als 2D-Array mit mehreren Zeilen.

    params_without_error : list
        Parameter ohne Fehlerangabe.
    """

    # if params_without_error is None:
    #     params_without_error = []

    # Sicherstellen, dass data 2D ist
    data = np.atleast_2d(np.array(data, dtype=float))

    n_rows = data.shape[0]
    n_params = len(params)

    # Array mit Struktur [p1, err_p1, p2, err_p2, ...]
    expanded = np.zeros((n_rows, 2 * n_params))

    # Daten in einheitliches Format überführen
    for row in range(n_rows):
        col_data = 0
```

```
        for i, p in enumerate(params):
            expanded[row, 2*i] = data[row, col_data]

            if p in params_without_error:
                expanded[row, 2*i + 1] = 0.0
                col_data += 1
            else:
                expanded[row, 2*i + 1] = data[row, col_data + 1]
                col_data += 2

    # Lambdify der Funktion
    f_num = sp.lambdify(params, function, "numpy")

    # Gaußsche Fehlerfortpflanzung
    errProne = [p for p in params if p not in params_without_error]
    absolut_err, relativ_err, err_params = gff(function, errProne, params_without_e
    err_func = sp.lambdify(err_params, absolut_err, "numpy")

    results = np.zeros((n_rows, 2))

    for row in range(n_rows):
        # Funktionswerte
        values = expanded[row, 0::2]
        func_value = f_num(*values)

    # Fehlerwerte in korrekter Reihenfolge
    err_args = []
    for p in params:
        idx = params.index(p)
        err_args.append(expanded[row, 2*idx])
        if p in params_without_error:
            err_args.append(0.0)
        else:
            err_args.append(expanded[row, 2*idx + 1])

    error_value = err_func(*err_args)
    results[row, 0] = func_value
    results[row, 1] = error_value

    # Parameter ohne Fehler aus Latex-Formeln entfernen
    for p in params_without_error:
        absolut_err = absolut_err.subs(f"thisWillTurnDelta{p.name}", 0)
        relativ_err = relativ_err.subs(f"thisWillTurnDelta{p.name}", 0)

    # Vereinfachte Funktionsdarstellung
    function = sp.simplify(function)

    print("Gegebene Funktion:")
    function_to_latex(function, "func", "func")

    print("\nAbsoluter Fehler:")
    function_to_latex(absolut_err, "errAbs", "errAbs")

    print("\nRelativer Fehler:")
    function_to_latex(relativ_err, "errRel", "errRel")
```

```
    return results
```

Export der Werte

Export als Tabelle

In [11]:
```
def table_to_latex(df_tex: str, texVarNameTab: str, texCom: str):
    """
    Fügt aus einem DataFrame entstandene Latex Tabelle der special Python File hinz

    Parameters
    ----------
    **df_tex** : pd.DataFrame.to_latex
        Uebersetztes DataFrame. Kann Label und Caption handlen. Dezimaltrennung ste

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Forme

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch tex

    Returns
    -------
    Schreibt automatisch in die sty file
    """

    # Hinzufuegen bzw. Ueberschreieben der Tabelle in die Sammlung
    with open(pyPath, 'r') as file:
        lines: List[str] = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines):
            if texVarNameTab in line: # Checkt, ob der Variablen Name bereits verge
                # print(f'{texVarName} starts at line {lineNum + 1}')
                found = True
                table_start_line = lineNum
                break

    if not found:
        print(f"Die neue Tabelle ({texVarNameTab}) wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n% " + texVarNameTab + "\n")
            file.write("\\newcommand{\\" + texCom + "}{" + df_tex + "}\n\n")
    else:
        print(f"Die alte Tabelle ({texVarNameTab}) wird überschrieben.")
        start_table = table_start_line + 1
        while start_table < len(lines) and lines[start_table].strip() == "":
            # Leere Zeile überspringen – das ist unser Marker
            start_table += 1
            break   # Wir wollen nur die erste leere Zeile
```

```
        # Falls kein leerer Marker gefunden wurde, gehen wir davon aus,
        # dass die Tabelle direkt in der nächsten Zeile startet.
        if start_table >= len(lines):
            start_table = table_start_line + 1

        end_table = start_table

        while end_table < len(lines):
            cur = lines[end_table].strip()

            # Abbruchbedingungen:
            if (
                cur.startswith(r"\\newcommand")
                or cur.startswith(r"\\section")
                or cur.startswith(r"\\subsection")
                or cur.startswith(r"\\begin")
            ):
                break

            end_table += 1

        if end_table > start_table:
            del lines[start_table - 1:end_table]

        insertion_point = table_start_line + 1  # Direkt nach der Zeile mit texVarN
        new_block = [
            # "\n\n",
            f"% Tabelle: {texVarNameTab}\n",
            f"\\newcommand{{\\{texCom}}}{{{df_tex}}}\n",
        ]
        lines[insertion_point:insertion_point] = new_block

        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

In [12]:

```
def combine_value_error(df: pd.DataFrame, show_series_values:bool=False, setTexTabC
    """Sucht im DataFrame nach einem Asdruck:
        "<name> [<unit>] <power>"
    und dem dazugehörigem Fehlerausdruck:
        "err <name>".
    Erstellt ein neues DataFrame für den Latex-Export. Kombiniert die Werte und Feh

    Parameters
    ----------
    **df* : pd.DataFrame
        Original data frame.

    **show_series_values** : bool
        Rather the table should show the experimental data or just its mean


    Returns
    -------
    pd.DataFrame
        Eine Kopie von ``df``.
    """
```

```
# Kopiert das alte DataFrame
df_tex = pd.DataFrame(index=df.index)

# Messwerte und ihre Ungenauigkeiten werden zudem in Arrays gespeichert, damit
value_error_arrays = {}

# def format_tex(row):
#     val = row[col]
#     err = row[err_col]
#     # return f'({val} \\pm {err}) \\mathrm{{{header_unit}}}'
#     val_r, err_r = round_sig_digs(err, val)
#     return f'${val_r} \\pm {err_r}$'

def format_tex(row, value_col, error_col):
    val = row[value_col]
    err = row[error_col]
    val_r, err_r = round_sig_digs(err, val)
    return f'${val_r} \\pm {err_r}$'

# ----------------------------
# 1) Einzelmessungen
# ----------------------------

# Einfaches Pattern fuer Messwerte, Einheiten und Exponenten:
pattern = re.compile(
    r'^\s*'
    r'(?P<name>[^\[]+?)'    # Name
    r'\s*\[\s*'
    r'(?P<unit>[^\]]*)'     # Einheit - * erlaubt auch leer
    r'\s*\]'
    r'(?:\s*10\^(?P<power>-?\d+))?'
    r'\s*$'
)

for col in df.columns:
    col_match = pattern.match(col)

    if not col_match:
        # No header that follows the expected schema - skip it
        # print(f'Skipping column {col!r}: not "<value> [<unit>] <power>"')
        continue

    # Extract the captured parts
    name = col_match.group('name').strip()
    unit = col_match.group('unit').strip()
    power = col_match.group("power")
    # print(f'Found measurement column: name={name!r}, unit={unit!r}, power={po

    if power:
        header_unit = f"$\\mathrm{{{unit}}} \\cdot 10^{{{power}}}$"
    else:
        header_unit = f"$\\mathrm{{{unit}}}$"

    # Build the name of the error column that must exist
    err_col = f'err {name}'
```

```
        if err_col not in df.columns:
            # print(f'No error column "{err_col}" for measurement "{name}" - skippi
            continue

        df_tex[f'{name} [{header_unit}]'] = df.apply(
            lambda row: format_tex(row, col, err_col),
            axis=1
        )

        # Speichert Werte im Array, damit diese für Berechnungen und Plotten besser
        values = df[col].to_numpy()
        errors = df[err_col].to_numpy()

        # Store in dictionary
        value_error_arrays[name] = {
            "values": values,
            "errors": errors
        }



    # ----------------------------
    # 2) Messreihena
    # ----------------------------

    series_pattern = re.compile(
        r'^(?P<base>\w+)_\d+'
        r'(?:\s*\[\s*(?P<unit>[^\]]+)\s*\])?'
    )

    grouped = {}

    for col in df.columns:
        series_match = series_pattern.match(col)
        if series_match:
            base = series_match.group("base")
            unit = series_match.group("unit") or ""
            grouped.setdefault(base, {"cols": [], "unit": unit})
            grouped[base]["cols"].append(col)


    for base, info in grouped.items():
        cols = info["cols"]
        series_unit = info["unit"] or ""
        err_col = f"err {base}"

        if err_col not in df.columns:
            continue

        values = df[cols]

        if calc_from_series:

            # Einzelwerte anzeigen
            if show_series_values:
                for ind, col in enumerate(cols, start=1):
```

```
                df_tex[f'${base}_{ind} [\\mathrm{{{series_unit}}}]$'] = df.appl
                    lambda row: format_tex(row, col, err_col),
                    axis=1
                )

        mean = values.mean(axis=1)
        std = values.std(axis=1, ddof=1)
        stat_err = std / np.sqrt(len(cols))
        sys_err = df[err_col]
        total_err = np.sqrt(stat_err**2 + sys_err**2)

        formatted = [
            f"${round_sig_digs(e, m)[0]} \\pm {round_sig_digs(e, m)[1]}$"
            for m, e in zip(mean, total_err)
        ]

        df_tex[f"$\\overline{{{base}}} [\\mathrm{{{series_unit}}}]$"] = formatt

        value_error_arrays[base] = {
            "values": mean.to_numpy(),
            "errors": total_err.to_numpy()
        }

    else:
        # Rohwerte speichern
        raw_values = df[cols].to_numpy()
        sys_err = df[err_col].to_numpy()
        errors = np.tile(sys_err.reshape(-1, 1), (1, len(cols)))

        value_error_arrays[base] = {
            "values": raw_values,
            "errors": errors
        }



# ----------------------------
# Export LaTeX
# ----------------------------
latex_table = df_tex.to_latex(
    decimal=',',
    caption=setTexTabCap,
    label=f"tab:{setTexTabLab}",
    position="h",
    index=False
)

table_to_latex(
    latex_table,
    texVarNameTab=tableVarNameTab,
    texCom=tableTexCom
)

return df_tex, value_error_arrays
```

In [13]:
```
def plot_with_xy_errorbars(
        x,
        y,
        xerr=None,
        yerr=None,
        xLabel=None,
        yLabel=None,
        title=None,
        label=None,
        fmt='o',
        capsize=4,
        color='blue',
        save_path=None,
        filename="plot",
        filetype="pdf",
        dpi=300,
        show_plot=True):
    """
    Plot data points with optional x and y error bars.

    Parameters
    ----------
    x : array-like or None
        X coordinates. If None, index positions are used.
    y : array-like
        Y coordinates.
    xerr : array-like, optional
        Errors of x values.
    yerr : array-like, optional
        Errors of y values.
    xLabel : str, optional
        Label for x-axis.
    yLabel : str, optional
        Label for y-axis.
    title : str, optional
        Plot title.
    label : str, optional
        Legend label.
    fmt : str
        Marker format (default 'o').
    capsize : float
        Error bar cap size.
    color : str
        Marker and error bar color.
    """

    y = np.asarray(y)

    if x is None:
        x = np.arange(len(y))
    else:
        x = np.asarray(x)

    plt.figure(figsize=(6, 4))

    plt.errorbar(
```

```
        x,
        y,
        xerr=xerr,
        yerr=yerr,
        fmt=fmt,
        capsize=capsize,
        color=color,
        label=label
    )

    if xLabel:
        plt.xlabel(xLabel)

    if yLabel:
        plt.ylabel(yLabel)

    if title:
        plt.title(title)

    if label:
        plt.legend()

    plt.grid(True)
    # plt.tight_layout()
    plt.legend(loc='best')

    # ✅ Save automatically
    if save_path is not None:
        os.makedirs(save_path, exist_ok=True)
        full_path = os.path.join(save_path, f"{filename}.{filetype}")
        plt.savefig(full_path, dpi=dpi)
        print(f"Plot saved to: {full_path}")

    if show_plot:
        plt.show()
    else:
        plt.close()
```

In [14]:

```
def process_experimental_data(
        # File Handling
        path=versuchsnummer + ".csv",
        setDelimiter=",",
        setHeader: int=0,
        setIndex_col: int=None,
        export_as_tex_tab: bool=True,
        name: str= versuchsnummer, # Einen kurzen aber präzisen und einzigartigen N

        # Latex Table Export
        show_series_values: bool=False,
        calc_from_series: bool=False,
        setTexTabCap: str="Experimentelle Daten",
        setTexTabLab: str="expData",
        tableTexCom: str=versuchsnummer + "ExpDataTab",

        # Plotting
        plot_data: bool=False,
```

```
        x=None,
        y=None,
        xerr=None,
        yerr=None,
        x_Label="x-Achse",
        y_label="y-Achse",
        title="Plot zum Versuch " + versuchsnummer,
        fmt='o',
        capsize=4,
        color='blue',
        save_path="../img/plot",
        filetype="pdf",
        dpi=300,
        show_plot=True,
    ):
    """
    Methode, die CSV-Dateien nach gewissen regeln ausliest. Dabei werden die folgen
        1. Wird eine TXT-Datei importiert, so soll geschaut werden, ob diese als CS

        2. Es wird gecheckt, ob eine Kopfzeile existiert, wenn ja, solld diese orde

        3. Es wird automatisiert erkannt, was der zu einem Wert gehörende Fehler is

        4. Es wird automatisch erkannt, falls dieselbe Messung wiederholt wurde und
        a. Es kann der Mittelwert bestimmt werden
        b. Es kann die Standardabweichung bestimmt werden.

        5. Der Export sowohl aller einzelnen Ergebnisse, als auch als gesamte Tabel

        6. Es wird automatisch erkannt, was die Einheiten sind und 10er Potenzen ko

        Note: Die meisten Funktionen funktionieren nur, wenn der Header ordentlich

    Parameter
    ---------
    **path**: Relatives verzeichnes der einzulesenden Datei. Am besten CSV-Datei ei

    **setDelimiter**: Default ist hier >> ; << (weil Excel mal wieder lost des Grau

    **setHeader**: Setzt fest, was die Header-Row ist. Default ist 0 (die oberste R
    """

    # Liest die Daten unmaipuliert
    try:
        df = pd.read_csv(path, delimiter=setDelimiter, header=0, index_col=None)
    except FileNotFoundError:
        print(f"The file {path} was not found.")

    # Wir erwarten nicht, dass zwei perfekt identische Zeilen existieren koennten,
    # Langfristig soll das noch verschönert werden und wirklich nur NaN Zeilen gelö
    no_NaN_data = df.drop_duplicates(keep= False)
    df.reset_index(drop=True, inplace=True)

    # Speichert dataFrame als Latex-Tabelle und fügt diese der python-results.sty h
    if export_as_tex_tab:
        df_tex, arrays = combine_value_error(
```

```
            no_NaN_data,
            show_series_values=show_series_values,
            setTexTabCap=setTexTabCap,
            setTexTabLab=setTexTabLab,
            tableVarNameTab=name + "Table",
            tableTexCom=tableTexCom,
            calc_from_series=calc_from_series,
)

    # Plotten der Daten mit Fehlerbalken, falls gewünscht. Es werden automatisch di
    if plot_data:
        if arrays is None:
            print("No arrays available for plotting.")
            return no_NaN_data, arrays

        keys = list(arrays.keys())

        # Auto-select first two datasets if not provided
        if x is None and len(keys) >= 1:
            x = arrays[keys[0]]["values"]
            xerr = arrays[keys[0]]["errors"]

        if y is None and len(keys) >= 2:
            y = arrays[keys[1]]["values"]
            yerr = arrays[keys[1]]["errors"]

        # If user passed column names as strings
        if isinstance(x, str):
            xerr = arrays[x]["errors"]
            x = arrays[x]["values"]

        if isinstance(y, str):
            yerr = arrays[y]["errors"]
            y = arrays[y]["values"]

        plot_with_xy_errorbars(
            x=x,
            y=y,
            xerr=xerr,
            yerr=yerr,
            xLabel=x_Label,
            yLabel=y_label,
            title=title,
            fmt=fmt,
            capsize=capsize,
            color=color,
            save_path=save_path,
            filename=name + '_' + aufgabe,
            filetype=filetype,
            dpi=dpi,
            show_plot=show_plot
        )

    return no_NaN_data, arrays, df_tex
```

In [26]:
```
def errf(function, param):
    error = 0
    parameters = []
    for par in param:
        d = sp.symbols('dstringdenniemandbenutzt' + par.name)
        partial = sp.diff(function, par) * d
        error = error + partial**2
        parameters.append((par,d))
    error_abs=sp.simplify(sp.sqrt(error),rational = True)
    error_rel=sp.simplify(sp.sqrt(error/function**2),rational = True)
    return(error_abs, error_rel, parameters)

#Hier soll eine Funktion entstehen, die eine Funktion auf einen Wert anwendet und g
#Erstmal für Eingabe der Daten in Form von 2d Arrays

"""
Darstellung einer Funktion in Latex
f: eine sympy Funktion
"""
def function_to_latex(f):
    print("Funktion:")
    display(Math(sp.latex(f,long_frac_ratio=2).replace('dstringdenniemandbenutzt',r
    print("Latex Code:")
    print(sp.latex(f,long_frac_ratio=2).replace('dstringdenniemandbenutzt',r'\Delta

"""
Berechnet Wert und Fehler für Daten
f: eine sympy Funktion
err: eine sympy Funktion, die den Fehler berechnet
data: Als array von Tupeln der Form [(a,da),(b,db),...] oder als array/liste [a,da,
    der Funktion genommen werden)
"""
def evaluate_with_error(f, err, values):
    if (np.ndim(values) != 1):                    #Falls der Input in mehrere Tupel
        values = np.concatenate(values)
    #print(values)
    result = f(*values[::2])
    uncertainty = err(*values)
    return result, uncertainty

"""
Gibt pro Zeile den Wert und den absoluten Fehler der Daten bzgl. der gegebenen Funk
Zusätzlich printet sie den Latex Code für die Formel, sowie die Fehlerformeln

Argumente:
function: sympy Funktion, in die die Messwerte eingesetzt werden sollen

params: Alle Parameter dieser Funktion als Array von Sympy-Zeichen

data: 2D-Array mit den Messdaten, sodass die Zeilen die Form haben: [Parameter 1, F
Die Funktion wird zeilenweise angewandt. Wird kein Fehler für einen Parameter angen
an die Funktion gegeben werden oder ganz weggelassen werden. Dann muss allerdings d

params_without_error: Alle Parameter zu denen kein Fehler explizit in den Daten ang
auch nicht in der Latex Form der Fehlerformel vor
"""
```

```python
def uncertainty(function,params, data, params_without_error=[]):
    #Expand data to include the uncertainty 0 for values with no assigned uncertain
    exp_data = np.zeros((data.shape[0],data.shape[1]+len(params_without_error)))
    i = 0      #läuft durch die Parameter
    j = 0      #läuft durch die expanded data
    z = 0      #läuft durch die eingegebene data
    while (i < len(params)):
        #print(exp_data)
        if (params[i] in params_without_error):
            exp_data[:,j] = data[:,z]
            i = i + 1
            j = j + 2
            z = z + 1
        else:
            exp_data[:,j] = data[:,z]
            exp_data[:,j+1] = data[:,z+1]
            i = i + 1
            j = j + 2
            z = z + 2
    #print(exp_data)
    #Create variable that stores parameters that have no assigned uncertainty
    params_with_error = []
    j = 0
    for n in np.arange(0,len(params)):
        if not (params[n] in params_without_error):
            params_with_error.append(params[n])
            j = j + 1

    #Get the given function and error function as numpy functions
    f = sp.lambdify(params,function)
    error_abs, error_rel, parameters = errf(function,params)
    err_abs = sp.lambdify(np.concatenate(parameters),error_abs) #Achtung Argumentre

    #Calculate the results for each row of data
    results = np.zeros((data.shape[0],2))
    for n in np.arange(0,data.shape[0]):
        results[n,:] = evaluate_with_error(f, err_abs, exp_data[n,:])
        #print(evaluate_with_error(f, err_abs, exp_data[n]))

    if (len(results) < 10):
        print("Results:")
        print(results)

    #Substitutes 0 for the uncertainty of the parameters without error, so it doesn
    for p in params_without_error:
        error_abs = error_abs.subs('dstringdenniemandbenutzt'+p.name,0)
    for p in params_without_error:
        error_rel = error_rel.subs('dstringdenniemandbenutzt'+p.name,0)

    #Prints out the Latex Code for the Functions
    function = sp.simplify(function,symbols = params, rational= True)
    function = sp.separatevars(function)
    function_to_latex(function)
    print("Fehler:")
    print("Absolut:")
    function_to_latex(error_abs)
```

```
    print("Relativ:")
    function_to_latex(error_rel)


    return(results)

"""
Macht das Selbe wie Uncertainty, akzeptiert aber die Daten in einer anderen Form.
data: Array, wobei jedes Element die Werte eines Parameters enthält (Reihenfolge [W
      Wert im Array kann entweder ein Array sein, oder nur ein einzelner Wert. Die
      Wird nur ein einzelner Wert angegeben, so ist dieser Parameter in allen Rechn
"""
def uncertainty2(function, params,data, params_without_error = []):
    length = 1
    for e in data:
        if(np.shape(np.array(e)) != ()):
            length = np.shape(np.array(e))[0]
            break
    data_2d = np.zeros((length,len(data)))
    for i in range (0,len(data)):
        data_2d[:,i] = data[i]
    return(uncertainty(function,params, data_2d,params_without_error = params_witho
```

Aufgabe 1: Clément-Desormes

In [27]:
```
aufgabe = "1"
```

In [28]:
```
# Werte einfügen
df, array, df_tex = process_experimental_data(
    path=versuchsnummer+ "-" +aufgabe + ".csv",
)

h1 = array["h1"]["values"]/100
h3 = array["h3"]["values"]/100

err_h1 = array["h1"]["errors"]/100
err_h3 = array["h3"]["errors"]/100

werte = np.column_stack([
    h1,
    err_h1,
    h3,
    err_h3
])

print(h1, err_h1)

# Aufbereten
h1, h3 = sp.symbols('h1 h3')
params = [h1, h3]
kappa = h1/(h1-h3)

results = uncertainty(kappa, params, werte, params_without_error=[])

results_df = pd.DataFrame(results, columns=["Kappa []", "err Kappa"])

results_df.to_latex(decimal=',', caption="Ergebnisse der Adiabatenkoeffizienten $\\

combine_value_error(results_df, setTexTabCap="Ergebnisse der Adiabatenkoeffizienten
```

```
Die alte Tabelle (221Table) wird überschrieben.
[0.043 0.04  0.046 0.045 0.043 0.043 0.045] [0.0014 0.0014 0.0014 0.0014 0.0014 0.00
14 0.0014]
Results:
[[1.34375    0.06068218]
 [1.37931034 0.06905935]
 [1.39393939 0.06145302]
 [1.36363636 0.05987285]
 [1.38709677 0.06503667]
 [1.38709677 0.06503667]
 [1.36363636 0.05987285]]
Funktion:
```

$$\frac{h_1}{h_1 - h_3}$$

```
Latex Code:
\frac{h_{1}}{h_{1} - h_{3}}
Fehler:
Absolut:
Funktion:
```

$$\sqrt{\frac{\Delta h_1^2 h_3^2 + \Delta h_3^2 h_1^2}{(h_1 - h_3)^4}}$$

```
Latex Code:
\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{\left(h_{1} - h
_{3}\right)^{4}}}
Relativ:
Funktion:
```

$$\sqrt{\frac{\Delta h_1^2 h_3^2 + \Delta h_3^2 h_1^2}{h_1^2 (h_1 - h_3)^2}}$$

```
Latex Code:
\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{h_{1}^{2} \left
(h_{1} - h_{3}\right)^{2}}}
Die neue Tabelle (Tabelle Aufgabe 1: Kappa der Einzelmessungen) wurde hinzugefügt
```

Out[28]:

```
(  Kappa [$\mathrm{}$]
 0     $1.34 \pm 0.06$
 1     $1.38 \pm 0.07$
 2     $1.39 \pm 0.06$
 3     $1.36 \pm 0.06$
 4     $1.39 \pm 0.07$
 5     $1.39 \pm 0.07$
 6     $1.36 \pm 0.06$,
 {'Kappa': {'values': array([1.34375   , 1.37931034, 1.39393939, 1.36363636, 1.387
09677,
          1.38709677, 1.36363636]),
   'errors': array([0.06068218, 0.06905935, 0.06145302, 0.05987285, 0.06503667,
          0.06503667, 0.05987285])}})
```

In [29]:

```python
# Alternative Fehler
df, array, df_tex = process_experimental_data(
    path= "A" + versuchsnummer+ "-" +aufgabe + ".csv",
)

h1 = array["h1"]["values"]/100
h3 = array["h3"]["values"]/100

err_h1 = array["h1"]["errors"]/100
err_h3 = array["h3"]["errors"]/100

werte = np.column_stack([
    h1,
    err_h1,
    h3,
    err_h3
])

print(h1, err_h1)

# Aufbereten
h1, h3 = sp.symbols('h1 h3')
params = [h1, h3]
kappa = h1/(h1-h3)

results = uncertainty(kappa, params, werte, params_without_error=[])

results_df = pd.DataFrame(results, columns=["Kappa []", "err Kappa"])
```

```
results_df.to_latex(decimal=',', caption="Ergebnisse der Adiabatenkoeffizienten $\\

combine_value_error(results_df, setTexTabCap="Ergebnisse der Adiabatenkoeffizienten

# new_err_k = np.sum(np.)
```

```
Die alte Tabelle (221Table) wird überschrieben.
[0.043 0.04  0.046 0.045 0.043 0.043 0.045] [0.00173205 0.00173205 0.00173205 0.0017
3205 0.00173205 0.00173205
 0.00173205]
Results:
[[1.34375    0.07507473]
 [1.37931034 0.08543879]
 [1.39393939 0.0760284 ]
 [1.36363636 0.07407344]
 [1.38709677 0.08046201]
 [1.38709677 0.08046201]
 [1.36363636 0.07407344]]
Funktion:
```

$$\frac{h_{1}}{h_{1} - h_{3}}$$

```
Latex Code:
\frac{h_{1}}{h_{1} - h_{3}}
Fehler:
Absolut:
Funktion:
```

$$\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{\left(h_{1} - h_{3}\right)^{4}}}$$

```
Latex Code:
\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{\left(h_{1} - h
_{3}\right)^{4}}}
Relativ:
Funktion:
```

$$\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{h_{1}^{2} \left(h_{1} - h_{3}\right)^{2}}}$$

```
Latex Code:
\sqrt{\frac{\Delta h_{1}^{2} h_{3}^{2} + \Delta h_{3}^{2} h_{1}^{2}}{h_{1}^{2} \left
(h_{1} - h_{3}\right)^{2}}}
Die neue Tabelle (Tabelle Aufgabe 1: Kappa der Einzelmessungen) wurde hinzugefügt
```

Out[29]:

```
(  Kappa [$\mathrm{}$]
 0     $1.34 \pm 0.08$
 1     $1.38 \pm 0.09$
 2     $1.39 \pm 0.08$
 3     $1.36 \pm 0.07$
 4     $1.39 \pm 0.08$
 5     $1.39 \pm 0.08$
 6     $1.36 \pm 0.07$,
 {'Kappa': {'values': array([1.34375   , 1.37931034, 1.39393939, 1.36363636, 1.387
09677,
          1.38709677, 1.36363636]),
   'errors': array([0.07507473, 0.08543879, 0.0760284 , 0.07407344, 0.08046201,
          0.08046201, 0.07407344])}})
```

In [30]:
```
mean_k = np.mean(results_df["Kappa []"])

mean_dk = 1/7 * np.sum(results_df["err Kappa"] ** 2 + np.std(results_df["err Kappa"

result = round_sig_digs(mean_dk, mean_k)

print(result)
```

```
(Decimal('1.374'), Decimal('0.007'))
```

Aufgabe 2: Rüchardt

In [31]:
```
aufgabe = "2"
```

In [32]:
```
# Werte einfügen
df, array, df_tex = process_experimental_data(
    path=versuchsnummer+ "-" +aufgabe + ".csv",
)

print(array)

## Deifinieren der Raumwerte

# Luftdruck in Pa
p0_val = 1013.3 * 100
err_p0 = 0.3 * 100

# Raumtemperatur in K
T_0 = 24.7 + 273.15
err_T_0 = 0.5

# Heidelberger Grvitationskonstante in m/s^2
g_hd = 9.80984
err_g_hd = 0.00002

## Definieren der Messwerte
n = 1 # n = 0: Luft, n = 1: Argon

# Massen der Schwingkörper in kg
m_val = array["m"]["values"][n]/1000
err_m = array["m"]["errors"][n]/1000

# Gasbehälter-Volumen in m^3
V_val = array["V"]["values"][n]/1000000
err_V = array["V"]["errors"][n]/1000000

# Radius des Rohres in m
r_val = array["2r"]["values"][n]/2000
err_r = array["2r"]["errors"][n]/2000

# Einzelmessungen der Periodendauer in s
t_i = array["t"]["values"][n]/75
err_t_i = array["t"]["errors"][n]/75
```

```
# Mittelwert der PEriodendauer in s
t_avg = np.mean(t_i)
err_t_avg = np.sqrt(np.sum(err_t_i**2 + np.std(t_i)**2))/len(t_i)

round_sig_digs(err_t_avg, t_avg)
```

```
Die alte Tabelle (221Table) wird überschrieben.
{'m': {'values': array([26.006, 26.116]), 'errors': array([0.002, 0.002])}, 'V': {'v
alues': array([5460, 5370]), 'errors': array([5, 5])}, '2r': {'values': array([15.9
7, 15.95]), 'errors': array([0.002, 0.002])}, 'p': {'values': array([1013.3, 101
3.3]), 'errors': array([0.3, 0.3])}, 'T': {'values': array([24.7, 24.7]), 'errors':
array([0.5, 0.5])}, 'g_hd': {'values': array([9.80984, 9.80984]), 'errors': array
([2.e-05, 2.e-05])}, 't': {'values': array([[75. , 74.8, 75. , 74.9, 74.9],
       [69. , 67.9, 66.9, 66.3, 67.9]]), 'errors': array([[0.3, 0.3, 0.3, 0.3, 0.3],
       [0.3, 0.3, 0.3, 0.3, 0.3]])}}
```

Out[32]: (Decimal('0.901'), Decimal('0.006'))

In [33]:

```
#
p0_sym, m_sym, g_sym, r_sym, pi_sym = sp.symbols('p_0 m g r pi')

pressures = p0_sym + m_sym * g_sym / (pi_sym * r_sym**2)

params = [p0_sym, m_sym, g_sym, r_sym, pi_sym]
# Definition der Messwerte und ihrer Fehler
messwerte = np.array([
    p0_val, err_p0,
    m_val, err_m,
    g_hd, err_g_hd,
    r_val, err_r,
    np.pi
])

print("p0:", p0_val, "+-", err_p0)
print("m:", m_val, "+-", err_m)
print("g:", g_hd, "+-", err_g_hd)
print("r:", r_val, "+-", err_r)
print("pi:", np.pi)
print("Messwerte: ", messwerte)

err_free = [pi_sym]

# print("params_without_error:", err_free)
# print("contains pi_sym?", pi_sym in err_free)

# results = pap_new(
#     pressures,
#     params,
#     messwerte,
#     err_free
# )
```

```
p0: 101330.0 +- 30.0
m: 0.026116 +- 2e-06
g: 9.80984 +- 2e-05
r: 0.007975 +- 1e-06
pi: 3.141592653589793
Messwerte:  [1.01330000e+05 3.00000000e+01 2.61160000e-02 2.00000000e-06
 9.80984000e+00 2.00000000e-05 7.97500000e-03 1.00000000e-06
 3.14159265e+00]
```

In [34]:

```
# Bestimmtung Kappa Rüchardt

#Definition von Symbolen und Parametern
T_sym, p_sym, V_sym= sp.symbols('T p V')
params = [m_sym,V_sym,r_sym,T_sym,p_sym]
function = 4*m_sym*V_sym/((r_sym**4)*(T_sym**2)*p_sym)
#Eingabe der Daten
werte = [m_val,err_m,V_val,err_V,r_val,err_r,t_avg,err_t_avg, p0_val,err_p0]
ph = uncertainty2(function,params,werte,params_without_error = [])
# K_a = ph[0][0]
# K_a_err = ph[0][1]

round_sig_digs(ph[0][1], ph[0][0])
```

```
Results:
[[1.68464836 0.02185002]]
Funktion:
```

$$\frac{4Vm}{T^2pr^4}$$

```
Latex Code:
\frac{4 V m}{T^{2} p r^{4}}
Fehler:
Absolut:
Funktion:
```

$$4\sqrt{\frac{1}{T^6p^4r^{10}}\left(T^2V^2\Delta p^2m^2r^2+16T^2V^2\Delta r^2m^2p^2+T^2p^2r^2\left(V^2\Delta m^2+\Delta V^2m^2\right)+4V^2\Delta T^2m^2p^2r^2\right)}$$

```
Latex Code:
4 \sqrt{\frac{1}{T^{6} p^{4} r^{10}} \left(T^{2} V^{2} \Delta p^{2} m^{2} r^{2} + 16
T^{2} V^{2} \Delta r^{2} m^{2} p^{2} + T^{2} p^{2} r^{2} \left(V^{2} \Delta m^{2} +
\Delta V^{2} m^{2}\right) + 4 V^{2} \Delta T^{2} m^{2} p^{2} r^{2}\right)}
Relativ:
Funktion:
```

$$\sqrt{\frac{\Delta m^2}{m^2}+\frac{\Delta p^2}{p^2}+\frac{16\Delta r^2}{r^2}+\frac{\Delta V^2}{V^2}+\frac{4\Delta T^2}{T^2}}$$

```
Latex Code:
\sqrt{\frac{\Delta m^{2}}{m^{2}} + \frac{\Delta p^{2}}{p^{2}} + \frac{16 \Delta r^
{2}}{r^{2}} + \frac{\Delta V^{2}}{V^{2}} + \frac{4 \Delta T^{2}}{T^{2}}}
```

Out[34]: (Decimal('1.6846'), Decimal('0.0219'))